SQL BASICS FOR DATABASE QUERYING & DATA MANAGEMENT

CASE STUDY

MyShop Sdn Bhd

RETAIL SALES ANALYSIS

PARTICIPANT WORKBOOK — Guided Exercises

Full End-to-End: Import → Schema → Core Queries → Advanced Analysis → Insights

200 Customers	100 Products	600 Orders	1,356 Items

Fakhrul Syahmi | Data Consultant & Certified Trainer | mufasyahcons@gmail.com

MySQL 8.0 • Malaysian Retail Dataset • © 2026

AGENDA

What We Will Cover Today

01 **Database Setup & Import**

Create schema · Import 4 CSV files · Verify data · Add indexes

02 **Schema Exploration**

DESCRIBE tables · Preview data · Distribution & quality checks

03 **Core SQL Tasks — 5 Queries**

WHERE · JOIN+GROUP BY · TOP-N · Date filter · Anti-Join

04 **Advanced Analysis — 8 Queries**

Monthly trend · Best sellers · CTE segmentation · CREATE VIEW

05 **Business Insights**

SQL results → business actions · Executive KPI · Practice exercises

DATASET

MyShop Sdn Bhd — Dataset Overview

200	100	600	1,356
Customers	**Products**	**Orders**	**Line Items**
20 Malaysian cities	10 categories	2020–2024	avg 2.26 per order

customers

PK: cutomers_id

customer_id · full_name · email · phone · city · join_date

products

PK: product_id

product_id · product_name · category · unit_price · stock_quantity · is_active

orders

PK: order_id

order_id · customer_id · order_date · status · payment_method · total_amount

order_items

PK: order_id

item_id · order_id · product_id · quantity · unit_price · discount · subtotal

customers —FK—> orders —FK—> order_items <—FK— products

PHASE 01

Database Setup & Data Import

- Create Schema
- Import CSVs
- Verify Rows
- Add Indexes

PHASE 1 · STEP 1

Create the Database & Tables

WHY THIS STEP MATTERS

We define the schema BEFORE importing data. CREATE TABLE sets column types, constraints, and foreign keys that enforce data integrity. Drop order follows FK dependencies: child tables first.

```
CREATE DATABASE IF NOT EXISTS myshop_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE myshop_db;
-- Drop in FK order (children first)
DROP TABLE IF EXISTS order_items; DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;   DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
  customer_id  INT           NOT NULL AUTO_INCREMENT,
  full_name    VARCHAR(100)  NOT NULL,
  email        VARCHAR(100)  NOT NULL,
  phone VARCHAR(20), city VARCHAR(60), join_date DATE,
  PRIMARY KEY (customer_id), UNIQUE KEY uk_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- Repeat for: products, orders, order_items (with FOREIGN KEY constraints)
SHOW TABLES;  -- Verify: 4 tables listed
```

KEY TERMS

AUTO_INCREMENT — MySQL auto-assigns the next ID
PRIMARY KEY — uniquely identifies each row; no NULLs, no duplicates
FOREIGN KEY — links child table to parent; enforces referential integrity
utf8mb4 — full Unicode; required for Malay/Arabic/Chinese data
ENGINE=InnoDB — supports transactions and foreign keys

NOTE: Create tables in this order: customers → products → orders → order_items. Reversing causes FK constraint errors.

PHASE 1 · STEPS 2–4

Import CSVs → Verify → Add Indexes

1	**myshop_customers.csv** No FK dependencies — import first	→ **customers**
2	**myshop_products.csv** No FK dependencies — import second	→ **products**
3	**myshop_orders.csv** Requires customers to exist	→ **orders**
4	**myshop_order_items.csv** Requires both orders & products	→ **order_items**

Import via Workbench: Right-click table → Table Data Import Wizard → Browse → Map columns → Execute

```
-- Verify row counts after import
SELECT 'customers' AS tbl, COUNT(*) AS rows FROM customers UNION ALL
SELECT 'products',COUNT(*) FROM products UNION ALL SELECT 'orders',COUNT(*) FROM orders
UNION ALL SELECT 'order_items',COUNT(*) FROM order_items;
-- Expected: 200 | 100 | 600 | 1356

-- Add performance indexes after import
CREATE INDEX idx_orders_date     ON orders(order_date);
CREATE INDEX idx_orders_status   ON orders(status);
CREATE INDEX idx_items_product   ON order_items(product_id);
CREATE INDEX idx_products_cat    ON products(category);
```

PHASE 02

Schema Exploration

- DESCRIBE tables
- Preview data
- Distribution analysis
- Data quality checks

PHASE 2

Know Your Data Before Writing Queries

WHY EXPLORE FIRST?

Skipping schema exploration is the #1 beginner mistake. You must understand data distributions and quality issues before writing business queries — or your results will be wrong.

```
DESCRIBE customers;          -- Inspect structure
SELECT * FROM orders LIMIT 10;  -- Preview rows (always use LIMIT)

-- Data distribution: customers per city
SELECT city, COUNT(*) AS n FROM customers GROUP BY city ORDER BY n DESC;

-- Order status breakdown
SELECT status, COUNT(*) AS n FROM orders GROUP BY status ORDER BY n DESC;

-- Payment method breakdown
SELECT payment_method, COUNT(*) FROM orders GROUP BY payment_method;

-- Date range
SELECT MIN(order_date) AS earliest, MAX(order_date) AS latest FROM orders;
```

DATA QUALITY CHECKS

```
-- NULLs in key columns:
SELECT SUM(CASE WHEN city IS NULL THEN 1 ELSE
0 END) AS null_cities FROM customers;

-- Duplicate emails:
SELECT email, COUNT(*) AS n FROM customers
GROUP BY email HAVING n > 1;

-- Orphaned items (no parent order):
SELECT COUNT(*) FROM order_items oi LEFT JOIN
orders o ON oi.order_id = o.order_id WHERE o.order_id
IS NULL;

-- Expected: 0 for all checks in this dataset
```

INSIGHT: After running the distribution queries, answer: (1) How many cities have customers? (2) What % of orders are Cancelled + Refunded? (3) Which payment method is most common?

PHASE 03

Core SQL Query Tasks

- Task 1: WHERE Filter
- Task 2: JOIN + GROUP BY
- Task 3: TOP-N LIMIT
- Task 4: Date Filter
- Task 5: Anti-Join

TASK 1 of 5

List All Customers from Kuala Lumpur

CONCEPT: SELECT + WHERE (String Filter)

WHERE filters rows by condition. Exact string match uses = . MySQL string comparisons are case-insensitive with utf8mb4_unicode_ci collation. Combine conditions with AND / OR.

```
USE myshop_db;

SELECT customer_id,
       full_name,
       email,
       phone
FROM   customers
WHERE  city = 'Kuala Lumpur'
ORDER  BY full_name ASC;

-- Try: change city, add AND join_date > '2022-01-01'
SELECT * FROM customers
WHERE  city = 'Kuala Lumpur'
  AND  join_date >= '2022-01-01';
```

Expected Output

```
+----+-----------------+--------------------------
-+
| id | full_name       | email
|
+----+-----------------+--------------------------
-+
| XX | Ahmad ...       | ahmad...@gmail.com
|
| XX | Lim ...         | lim...@yahoo.com
|
+----+-----------------+--------------------------
-+
  ~25-35 rows returned
```

PRACTICE EXERCISES

1. List customers from Penang
2. Find customers who joined after '2022-01-01'
3. Combine: city='Penang' AND join_date > '2022-01-01'
4. Count them: SELECT COUNT(*) FROM customers WHERE city='Kuala Lumpur'

INSIGHT: WHERE clause tip: put the most selective (fewest rows) condition first. For indexed columns, equality (=) is always faster than LIKE with a leading wildcard (%text).

TASK 2 of 5

Total Revenue Generated Per Product Category

CONCEPT: Multi-Table JOIN + GROUP BY + SUM()

JOIN combines rows using a matching key. GROUP BY collapses rows by a column so aggregate functions (SUM, COUNT, AVG) can compute per-group results. Always filter status='Completed' to exclude cancelled revenue.

```
USE myshop_db;

SELECT   p.category,
         COUNT(DISTINCT o.order_id)     AS total_orders,
         SUM(oi.quantity)               AS units_sold,
         ROUND(SUM(oi.subtotal), 2)     AS net_revenue,
         ROUND(SUM(oi.discount), 2)     AS total_discount
FROM     order_items  oi
JOIN     products     p  ON oi.product_id = p.product_id
JOIN     orders       o  ON oi.order_id   = o.order_id
WHERE    o.status = 'Completed'
GROUP BY p.category
ORDER BY net_revenue DESC;
```

JOIN PATH EXPLAINED

We JOIN 3 tables:
1. order_items (the data source)
2. products → to get the category name
3. orders → to filter by status

Why start from order_items?
It is the most granular table — one row per product per order. From there, we branch to related tables to get the labels and filters we need.

INSIGHT: Rule: Every column in GROUP BY must also appear in SELECT (not inside an aggregate). Every SELECT column not in an aggregate MUST be in GROUP BY.

TASK 3 of 5

Top 5 Customers by Total Spending

CONCEPT: ORDER BY DESC + LIMIT = Top-N Pattern

LIMIT restricts the number of rows returned. Combined with ORDER BY DESC, this is the standard Top-N pattern used in every leaderboard, dashboard, and report.

```
USE myshop_db;

SELECT   c.customer_id,
         c.full_name,
         c.city,
         COUNT(o.order_id)             AS total_orders,
         ROUND(SUM(o.total_amount), 2) AS total_spent,
         ROUND(SUM(o.total_amount) /
               COUNT(o.order_id), 2)   AS avg_order_value
FROM     customers c
JOIN     orders    o ON c.customer_id = o.customer_id
WHERE    o.status = 'Completed'
GROUP BY c.customer_id, c.full_name, c.city
ORDER BY total_spent DESC
LIMIT    5;
```

PRACTICE EXERCISES

1. Change LIMIT 5 to LIMIT 10
2. Add HAVING total_orders >= 3 after GROUP BY
3. Remove WHERE status='Completed' — does the top spender change?
4. Sort by avg_order_value DESC — who places the biggest individual orders?
5. Add c.city to see where top spenders are located

INSIGHT: GROUP BY rule: include ALL non-aggregate SELECT columns. Missing one gives wrong results silently.

TASK 4 of 5

Orders Placed in Q1 2024 (Jan – Mar)

CONCEPT: Date Filtering — BETWEEN vs Date Functions

BETWEEN is inclusive on both ends. On an indexed date column, BETWEEN allows MySQL to use the index (fast). Functions like YEAR() or MONTH() applied to a column prevent index use — always slower on large datasets.

```
USE myshop_db;

-- Method A: BETWEEN (fast — uses date index)
SELECT order_id, customer_id, order_date,
       status, total_amount
FROM   orders
WHERE  order_date BETWEEN '2024-01-01' AND '2024-03-31'
ORDER  BY order_date ASC;

-- Method B: QUARTER() function (readable, slightly slower)
SELECT order_id, order_date, total_amount
FROM   orders
WHERE  YEAR(order_date) = 2024 AND QUARTER(order_date) = 1;

-- Q1 2024 Summary
SELECT COUNT(*) AS orders,
  SUM(CASE WHEN status='Completed' THEN total_amount ELSE 0 END) AS q1_revenue,
  COUNT(CASE WHEN status='Cancelled' THEN 1 END) AS cancelled
FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
```

DATE FUNCTIONS REFERENCE

YEAR(date) => 2024
MONTH(date) => 3
QUARTER(date) => 1
DATE_FORMAT(date,'%Y-%m') => 2024-03
MONTHNAME(date) => March
DATEDIFF(d1,d2) => days between
CURDATE() => today
NOW() => current datetime

NOTE: BETWEEN is inclusive: BETWEEN '2024-01-01' AND '2024-03-31' includes both Jan 1 and Mar 31.

TASK 5 of 5

Customers Who Have Never Placed an Order

CONCEPT: LEFT JOIN Anti-Join Pattern

LEFT JOIN returns ALL rows from the left table + matching rows from the right. Where no match exists, right-side columns are NULL. Adding WHERE right.id IS NULL gives rows from the left with NO match — the Anti-Join pattern.

```
USE myshop_db;

-- Method A: LEFT JOIN Anti-Join (recommended)
SELECT c.customer_id, c.full_name,
       c.email, c.city, c.join_date
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE  o.order_id IS NULL
ORDER  BY c.join_date DESC;

-- Method B: NOT IN subquery
SELECT customer_id, full_name, city
FROM   customers
WHERE  customer_id NOT IN (
    SELECT DISTINCT customer_id FROM orders);

-- Summary: inactive rate
SELECT COUNT(*) AS total,
  SUM(CASE WHEN o.order_id IS NULL THEN 1 ELSE 0 END) AS never_ordered,
  ROUND(SUM(CASE WHEN o.order_id IS NULL THEN 1 ELSE 0 END)/COUNT(*)*100,1) AS
pct
FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

LEFT JOIN vs INNER JOIN

INNER JOIN: rows that match in BOTH tables only

LEFT JOIN: ALL left rows + matching right rows; NULL where no match

Anti-Join: LEFT JOIN + WHERE right.id IS NULL
→ rows in left table with NO match in right table

Common mistake: using INNER JOIN when you need LEFT JOIN — silently drops customers with no orders

INSIGHT: Business value: never-ordered customers are re-engagement opportunities. Export this list for a first-purchase discount email campaign.

PHASE 04

Advanced Analysis Queries

A1: Monthly Trend	A2: Best Sellers	A3: Segmentation (CTE)	A4: Discount Impact
A5: City Revenue	A6: Payment	A7: Cancellation	A8: CREATE VIEW

ANALYSIS 1 + 2

Monthly Revenue Trend & Best-Selling Products

A1 — Monthly Revenue Trend

```
SELECT
  DATE_FORMAT(order_date,'%Y-%m') AS month,
  COUNT(DISTINCT order_id)        AS orders,
  ROUND(SUM(total_amount),2)      AS revenue,
  ROUND(AVG(total_amount),2)      AS avg_value
FROM orders WHERE status='Completed'
GROUP BY DATE_FORMAT(order_date,'%Y-%m'),
  YEAR(order_date), MONTH(order_date)
ORDER BY YEAR(order_date), MONTH(order_date);
```

INSIGHT: A1: Look for seasonal peaks (Nov/Dec year-end, April/May Hari Raya). Declining avg_order_value with rising order count signals a shift to lower-priced products.

A2 — Best-Selling Products

```
SELECT p.product_name, p.category,
  SUM(oi.quantity)            AS units_sold,
  ROUND(SUM(oi.subtotal),2)   AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN orders   o ON oi.order_id   = o.order_id
WHERE o.status = 'Completed'
GROUP BY p.product_id, p.product_name, p.category
ORDER BY revenue DESC LIMIT 15;
```

INSIGHT: A2: High unit sales ≠ high revenue. A RM9.90 item selling 500 units = RM4,950 vs a RM299 item selling 50 = RM14,950. Always compare both metrics.

PRACTICE

A1: Add ORDER BY revenue DESC LIMIT 1 to find the single best month.
A2: Sort by units_sold DESC instead — does the top revenue product also top unit sales?
A3: Add WHERE p.category='Electronics' to focus on one category.

ANALYSIS 3

Customer Segmentation — CTE + CASE WHEN

NEW CONCEPT: WITH (CTE — Common Table Expression)

A CTE creates a named temporary result set you can reference like a table. CTEs break complex queries into readable steps. They are not stored — they exist only for that query execution.

```
WITH customer_orders AS (
  SELECT c.customer_id, c.full_name, c.city,
         COUNT(o.order_id)       AS order_count,
         SUM(o.total_amount)     AS lifetime_value
  FROM   customers c
  LEFT JOIN orders o ON c.customer_id = o.customer_id
    AND o.status = 'Completed'
  GROUP BY c.customer_id, c.full_name, c.city
)
SELECT customer_id, full_name, city,
       order_count,
       ROUND(lifetime_value,2) AS ltv,
       CASE
         WHEN order_count = 0  THEN 'Inactive (Never Ordered)'
         WHEN order_count = 1  THEN 'One-Time Buyer'
         WHEN order_count <= 3 THEN 'Occasional Buyer'
         WHEN order_count <= 6 THEN 'Regular Customer'
         ELSE                       'Loyal / VIP'
       END AS segment
FROM customer_orders
ORDER BY order_count DESC;
```

SEGMENTS & BUSINESS ACTIONS

Inactive → re-engagement email with first-order discount
One-Time → win-back offer; highlight new products
Occasional → loyalty nudge, email sequence
Regular → reward with points / perks program
VIP/Loyal → exclusive tier, early access, gifts

This is RFM-lite segmentation — a foundation of CRM analytics.

ANALYSIS 4–7

Discount Impact · City Revenue · Payments · Cancellation

```
-- A4: Discount impact by product
SELECT p.category, p.product_name,
  ROUND(SUM(oi.discount),2)                          AS
total_discount,
  ROUND(SUM(oi.discount)/
    (SUM(oi.subtotal)+SUM(oi.discount))*100,1)      AS
margin_eroded_pct
FROM order_items oi JOIN products p ON
oi.product_id=p.product_id
JOIN orders o ON oi.order_id=o.order_id WHERE
o.status='Completed'
GROUP BY p.category, p.product_name HAVING total_discount > 0
ORDER BY total_discount DESC LIMIT 10;
```

```
-- A5: Revenue by city
SELECT c.city,
  COUNT(DISTINCT c.customer_id)      AS customers,
  ROUND(SUM(o.total_amount),2)        AS revenue,
  ROUND(SUM(o.total_amount)/
    COUNT(DISTINCT c.customer_id),2)  AS rev_per_customer
FROM customers c JOIN orders o ON c.customer_id=o.customer_id
WHERE o.status='Completed'
GROUP BY c.city ORDER BY revenue DESC;
```

```
-- A6: Payment method analysis
SELECT payment_method,
  COUNT(*) AS orders,
  ROUND(AVG(total_amount),2) AS avg_value
FROM orders WHERE status='Completed'
GROUP BY payment_method ORDER BY orders DESC;
```

```
-- A7: Cancellation rate by city
SELECT c.city, COUNT(o.order_id) AS total,
  ROUND(SUM(CASE WHEN o.status IN
    ('Cancelled','Refunded') THEN 1 ELSE 0 END)
    /COUNT(o.order_id)*100,1) AS cancel_pct
FROM customers c JOIN orders o ON c.customer_id=o.customer_id
GROUP BY c.city HAVING total>=5 ORDER BY cancel_pct DESC
LIMIT 8;
```

INSIGHT: A4: margin_eroded_pct > 15% → discount destroying profitability. Cap at 10% for premium SKUs. A5: High rev_per_customer in a city = strong buyers; low orders = untapped market.

INSIGHT: A6: If Credit Card avg_value > E-Wallet, premium buyers prefer cards. Offer instalment plans for items > RM200. A7: Cities > 12% cancel rate may have delivery/logistics issues.

ANALYSIS 8

Create a Reusable Sales Summary VIEW

WHAT IS A VIEW?

A VIEW is a saved SELECT query stored as a named virtual table. Once created, query it like any table — no need to rewrite the complex JOIN logic. Perfect for recurring reports and BI tool connections (Power BI, Looker, Excel).

```
CREATE OR REPLACE VIEW vw_sales_summary AS
SELECT
  o.order_id, o.order_date,
  DATE_FORMAT(o.order_date,'%Y-%m')  AS order_month,
  YEAR(o.order_date)                 AS order_year,
  QUARTER(o.order_date)              AS quarter,
  o.status, o.payment_method, o.total_amount,
  c.customer_id, c.full_name AS customer_name, c.city,
  p.product_id, p.product_name, p.category,
  oi.quantity, oi.unit_price, oi.discount,
  oi.subtotal AS line_total
FROM   orders o
JOIN   customers   c  ON o.customer_id = c.customer_id
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products    p  ON oi.product_id = p.product_id;

-- Use the view (complex JOIN reduced to 3 lines!)
SELECT category, SUM(line_total) AS revenue
FROM   vw_sales_summary WHERE status='Completed'
GROUP  BY category ORDER BY revenue DESC;
```

VIEW vs TABLE + PRACTICE

TABLE: stores data on disk permanently
VIEW: runs saved query on-demand; always current; no data stored

Verify: SHOW FULL TABLES WHERE Table_type = 'VIEW';

PRACTICE:
1. Use the view to get monthly 2023 revenue
2. Find the top city by line_total from the view
3. Filter view for Electronics only: WHERE category='Electronics'

PHASE 05

Business Insights & Recommendations

SQL → Action Framework

Executive KPI Query

Practice Exercises

PHASE 5

SQL Results → Business Actions

SQL Finding	Business Insight	Action
X% inactive customers	High dormancy = lost acquisition spend	**Reactivation email + first-order discount**
Top 5 = Y% of revenue	Revenue concentration risk	**VIP loyalty tier with exclusive perks**
Electronics = #1 revenue	High-value category driving revenue	**More inventory + homepage feature**
Q4 revenue spike	Clear seasonal demand peak	**Pre-load stock 6 weeks before Q4**
Cancel rate > 8%	Post-purchase abandonment signal	**Survey cancelled customers; check logistics**
Credit Card: highest avg value	Premium buyers prefer credit cards	**Instalment plans for products > RM200**

PHASE 5

Executive KPI Summary Query

```
-- MyShop Sdn Bhd | Executive KPI Dashboard — Single Row Output
SELECT
  (SELECT COUNT(*) FROM customers)                    AS total_customers,
  (SELECT COUNT(DISTINCT customer_id) FROM orders
   WHERE status = 'Completed')                        AS active_buyers,
  (SELECT COUNT(*) FROM customers c LEFT JOIN orders o
   ON c.customer_id=o.customer_id WHERE o.order_id IS NULL) AS never_ordered,
  (SELECT COUNT(*) FROM orders)                       AS total_orders,
  (SELECT COUNT(*) FROM orders WHERE status='Completed')    AS completed,
  (SELECT COUNT(*) FROM orders
   WHERE status IN ('Cancelled','Refunded'))          AS cancelled_refunded,
  (SELECT ROUND(SUM(total_amount),2) FROM orders
   WHERE status='Completed')                          AS total_revenue_rm,
  (SELECT ROUND(AVG(total_amount),2) FROM orders
   WHERE status='Completed')                          AS avg_order_value_rm,
  (SELECT COUNT(*) FROM products WHERE is_active=1)   AS active_products,
  (SELECT MIN(order_date) FROM orders)                AS data_from,
  (SELECT MAX(order_date) FROM orders)                AS data_to;
```

CHALLENGE EXERCISES

Advanced challenges:

1. Rank customers by spending within each city using RANK() OVER (PARTITION BY city ORDER BY SUM(total_amount) DESC)
2. Calculate month-over-month revenue growth % using LAG() window function
3. Find products frequently bought together: self-join on order_items
4. Days between first and last order per customer: DATEDIFF(MAX(order_date), MIN(order_date))

Session Complete! Well Done.

What You Have Achieved

- ✓ Built a MySQL database from scratch with correct schema & FK constraints
- ✓ Imported 2,256 rows across 4 related tables in the correct order
- ✓ Explored schema and validated data quality before writing queries
- ✓ Written 13+ SQL queries from basic SELECT to CTEs and CREATE VIEW
- ✓ Translated SQL results into actionable business recommendations

Fakhrul Syahmi | Data Consultant & Certified Trainer